A Middleware-Centric Architectural Digital Model for Pre-Deployment Verification and CI/CD Gating in Naval Combat Management Systems

Ibrahim Onuralp Yigit
Command Control and Defense Technologies, HAVELSAN Inc.
Istanbul, Turkiye
iyigit@havelsan.com.tr

Industrial Co-Authors & CMS Architects
Naval Combat Systems Directorate, HAVELSAN Inc.
Ankara, Turkiye
contact@havelsan.com.tr

Abstract

Modern mission-critical Naval Combat Management Systems (CMS), such as HAVELSAN's combat-proven ADVENT CMS, are complex distributed real-time systems built upon publish/subscribe (pub/sub) middleware. They integrate hundreds of software applications across shipboard operator consoles (OPCONs), radar trackers, weapon controllers, tactical data links (Link 11, Link 16, Link 22, Link H), and multi-domain platforms (surface combatants, amphibious assault flagships, maritime patrol aircraft, coastal surveillance stations, and unmanned vessels). In these safety-critical environments, the most severe integration defects are non-local architectural misconfigurations: incompatible Quality-of-Service (QoS) contracts on hard-deadline weapon assignment channels, overlapping CPU core affinity masks on multi-core tactical servers, orphaned tactical topics, and circular package dependencies across engagement planning modules. These defects easily survive isolated unit and component testing, only to surface during costly shipyard integration, live-fire sea trials, or multi-ship joint tactical exercises.

We report on an architectural digital model generated using System as a Graph (SaaG), an industrial framework designed for pre-deployment verification and continuous integration and delivery (CI/CD) gating in naval combat management systems. Reconstructed fresh from candidate release descriptors rather than relying on runtime synchronization, SaaG constructs an attributed typed multigraph ($G = (V, E, \tau_V, \tau_E, w_V, w_E)$) capturing applications, brokers, tactical topics, tactical processing nodes, and shared libraries. The framework derives asymmetric failure-dependency projections from pub/sub communication topologies ($B \xrightarrow{\text{DEPENDS\_ON}} A$) and gates the CI/CD pipeline against static rule violations mapped to military safety severity levels (MIL-STD-882E S1–S4).

Our central industrial finding is clear: graph analysis is not the computational bottleneck. Across five scaling operational naval platform profiles (1,498 to 3,254 components across up to 66 distributed nodes), static verification executes in merely 0.125 to 1.152 s (mean 1.152 s, P95 1.161 s on the 66-node LHD flagship; 0.760 s on a frigate combatant), representing under 0.22% of a typical 9-minute automated build pipeline. We demonstrate that the SaaG framework statically detects critical pub/sub misconfiguration scenarios prior to release packaging without requiring expensive physical testbeds. However, an industrial gap analysis reveals that of seven verification capabilities specified with naval combat system architects, six remain constrained by configuration-data acquisition across defense engineering silos rather than by algorithmic limits. We report the framework architecture, multi-scale verification performance, representative misconfiguration case studies, and practical lessons from industrial deployment.

CCS Concepts

• **Computer systems organization → Reliability**; • **Software and its engineering → Software defect analysis**.

Keywords

Naval Combat Management Systems (CMS), ADVENT CMS, SaaG Framework, Architectural Digital Model, Middleware Verification, Pub/Sub QoS Contracts, CI/CD Gating, CPU Core Pinning, Static Rule Auditing, OMG DDS, MIL-STD-882E, Network Enabled Capability (NEC)

1 Introduction & Industrial Problem Statement

Continuous Integration and Continuous Delivery (CI/CD) pipelines have fundamentally transformed software engineering by automating compilation, testing, and artifact generation [15]. In naval defense systems, modern Combat Management Systems (CMS) have evolved from monolithic mainframes into modular, distributed, open-architecture systems governed by high-performance publish/subscribe (pub/sub) middleware [6].

1.1 The Pre-Deployment Verification Gap in Naval Combat Systems

Modern naval operations require rapid sensor-to-shooter loops, multi-sensor data fusion, coordinated weapon assignment, and seamless cross-platform interoperability. Built to fulfill these demands, **ADVENT CMS** (developed by **HAVELSAN** in cooperation with the Turkish Naval Forces Research Center Command—ARMERKOM [1]) represents a state-of-the-art naval combat suite deployed across a wide product tree:

- **ADVENT Kalyon:** Surface combatant CMS encompassing frigates, corvettes, and specialized mine countermeasures (MCM) vessels.
- **Milgem CMS:** Combat management suite for Ada-class corvettes and Istanbul-class frigates, handling multi-threat warfare (AAW, ASuW, ASW, EW).
- **LHD CMS:** Specialized fleet flagship configuration for Landing Helicopter Dock amphibious assault ships (e.g., L400 TCG Anadolu), providing task group command oversight,

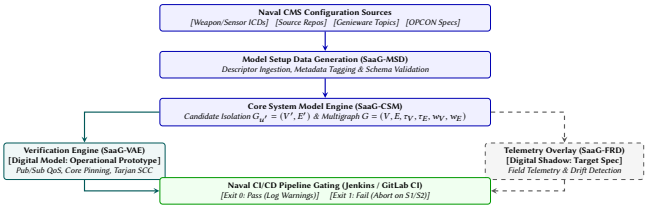

Figure 1: SaaG Architectural Digital Model Pipeline Integration Overview for ADVENT Combat Management Systems. (Dashed boxes denote specified capabilities not yet fully integrated into prototype).

multi-link gateway centers, joint operations planning, and unmanned vehicle swarm control.

- **ADVENT Rota:** Mission Management System (MMS) for unmanned surface, underwater, and aerial vehicles (USV/UUV/UAV), integrating STANAG 4586 compliance [3], autonomous navigation, and direct payload control from shipboard consoles without dedicated GCS hardware.
- **ADVENT Martı:** Airborne Command and Control System designed for Maritime Patrol Aircraft (MPA) and naval helicopters, executing airborne surveillance, sonobuoy processing, and tactical data link relay.
- **ADVENT Ufuk:** Land-based coastal surveillance and C2 information management system aggregating coastal radar networks, AIS, ADS-B, and electro-optical sensors to compile and disseminate the Recognized Maritime Picture (RMP).

Across these platforms, ADVENT CMS comprises over **155 external system integrations**, **750 distributed software applications**, and **13,000,000 lines of code**. Subsystems communicate over **Genieware**—a national publish/subscribe middleware designed for real-time mission-critical defense systems, operating on a DDS-like pub/sub architecture [6]—and Tactical Data Links (TDLs: Link 11, Link 16 [4], Link 22 [5], and ADVENT Native Link H). Under the **Network Enabled Capability (NEC)** paradigm, ADVENT shares sensor tracks and weapon allocations across platforms, enabling Force-Wide Weapon/Sensor Allocation (WASA) and remote firing authorizations.

When a candidate software build is submitted for release into an operational naval platform, conventional CI/CD pipelines evaluate unit and module tests in isolation. Consequently, critical architectural misconfigurations slip through:

- **Hardware CPU Core Contention on OPCONs:** Latency-critical track fusion daemons or fire control calculators inadvertently pinned to overlapping CPU core affinity masks with non-real-time GUI renderers or background loggers on multi-core tactical consoles.
- **Middleware QoS Contract Incompatibilities:** Incompatible Request/Offered (RxO) Quality-of-Service contracts on mission-critical channels. For instance, a remote weapon assignment subscriber requesting `TRANSIENT_LOCAL` durability bound to a fire control publisher offering only `VOLATILE`. In pub/sub middleware like Genieware and DDS, endpoints with incompatible QoS contracts never match, so data never flows; the middleware signals this through incompatible QoS status notifications, but these are rarely trapped in operational code, making the failure effectively silent.
- **Silent Tactical Topic Disconnections:** Software units publishing to or consuming from topics with schema definitions that diverge across platform releases, or refactored TDL forwarding topics left with zero active subscribers.
- **Circular Package Dependencies:** Transitive dependency cycles between weapon allocation planners (WASA) and threat evaluation modules that manifest as initialization deadlocks during OPCON console start-up.

Provisioning full target hardware testbeds (physical multi-console Combat Information Centers [CIC], real sensor/weapon simulators, and multi-ship test ranges) for every candidate release is prohibitively expensive, logistically complex, and slow. Leaving architectural defects to be discovered during shipyard commissioning, harbor acceptance tests (HAT), or sea acceptance tests (SAT) leads to massive project delays and severe safety risks.

1.2 The Architectural Digital Model Paradigm & Dual Operational Usages

To eliminate this pre-deployment gap and manage system complexity across the lifecycle, we developed **System as a Graph (SaaG)**. Rather than relying on heavyweight runtime testbeds, SaaG constructs an attributed graph model of the target combat system topology directly from configuration artifacts and interface contracts.

In accordance with the digital twin classification of Kritzinger et al. [7]:

- A **Digital Model** has no automated bidirectional data exchange with the physical system; the digital representation is constructed from static configuration artifacts.
- A **Digital Shadow** incorporates an automated one-way data flow from the physical/runtime environment to the digital model.
- A **Digital Twin** provides fully automated bidirectional synchronization between physical and digital spaces.

Under this taxonomy, our operational framework is strictly an *architectural digital model*—reconstructed deterministically from configuration artifacts. In naval engineering practice, the architectural digital model serves two complementary operational usages: (1) **Pre-Deployment Verification & CI/CD Gating**, preventing non-conforming builds ($G_{u'}$) from reaching integration testbeds; and (2) **Architectural Improvement & Refactoring of Existing Platforms**, enabling architects to detect circular dependency cycles, discover criticality bottlenecks, prune dead topic configurations, and perform "what-if" impact simulations across fielded combat systems.

1.3 Key Industrial Contributions & Empirical Findings

This paper makes the following contributions:

(1) **Naval CMS Architectural Model Baseline (§2):** A formal directed multigraph representation ($G = (V, E, \tau_V, \tau_E, w_V, w_E)$)

capturing 5 entity classes, 6 structural relations, and 6 failure-dependency projection rules that explicitly map the downstream propagation of architectural risk in pub/sub naval combat systems.

(2) **CI/CD Pipeline Gating & Verification Methodology (§3, §4):** A two-stage deployment gate operating on standard CI runners with a defined safety severity rubric (S1–S4) aligned with military safety frameworks (MIL-STD-882E [2]). We demonstrate pre-deployment static verification across representative mission-critical pub/sub misconfiguration scenarios (QoS mismatches, CPU core contention, topic continuity breaks, circular dependencies).

(3) **Verification Complexity & Industrial Bottleneck Analysis (§5, §6):** Empirical benchmarks across 5 scaling operational naval platform profiles (1,498 to 3,254 components; 1 to 66 nodes) modeled on ADVENT CMS platform lines, demonstrating that static verification executes in 0.125 s (Martı), 0.410 s (Ufuk), 0.435 s (Rota), 0.760 s (Milgem frigate), and 1.152 s (LHD flagship with 66 nodes, P95 1.161 s), consuming $< 0.22\%$ of a 9-minute build. We analyze why physical node distribution drives graph lifting complexity, and document that 6 of 7 specified verification capabilities are blocked by configuration data acquisition across defense engineering silos, not by graph analysis complexity.

2 The Architectural Digital Model: Formulation and Graph Derivation

SaaG adopts and operationalizes the formal graph modeling foundation established by Yigit and Buzluca [17] for distributed publish-subscribe systems. We formalize the distributed naval CMS middleware topology as an attributed, weighted directed multigraph:

$$G = (V, E, \tau_V, \tau_E, w_V, w_E) \tag{1}$$

2.1 Entity Classes and Structural Relations

The vertex set V is partitioned into five distinct entity types ($\tau_V : V \to \mathcal{T}_v$):

- **Applications (V_{app}):** Executable CMS binaries (e.g., track fusion gateway `track-fusion-gw`, threat evaluation service `threat-eval-srv`, WASA planner `wasa-allocator`, tactical display server `geodisplay-srv`, console client `opcon-ui`, Link 16 gateway `advlink-l16`, USV controller `rota-autonomy`).
- **Brokers (V_{broker}):** Genieware discovery domains, routing daemons, and middleware gateways facilitating inter-node tactical communication.
- **Topics (V_{topic}):** Typed publish/subscribe communication channels carrying domain payloads (e.g., `tactical.tracks`, `weapon.assignment.cmd`, `threat.eval.result`, `link16.jseries`, `sensor.radar.plots`, `mine.qroutes`).
- **Infrastructure Nodes (V_{node}):** Physical OPCON console workstations, ruggedized VME/VPX server chassis, and mission computers characterized by CPU core capacity $C(v_p)$ and memory bounds.
- **Libraries (V_{lib}):** Shared dynamic libraries, STANAG 4586 parsers, coordinate conversion routines, tactical math libraries, and NATO symbology decoders.

Six structural edge types ($\tau_E : E_{\text{structural}} \to \mathcal{T}_e$) are extracted directly from configuration descriptors: `PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, and `USES`.

2.2 Asymmetric Failure-Dependency Projection

In pub/sub middleware, data messages flow from publisher to subscriber ($A \to B$). However, **structural failure dependency points in the exact opposite direction ($B \xrightarrow{\text{DEPENDS\_ON}} A$)**: if Publisher A (e.g., radar tracker) fails or produces corrupt messages, Subscriber B (e.g., fire control calculator) is starved or degraded; conversely, if Subscriber B crashes, Publisher A continues unaffected.

Building upon the dependency analysis methodology of Yigit and Buzluca [17], SaaG projects logical `DEPENDS_ON` edges from structural topology (Table 1).

Table 1: Derived Logical Dependency Edge Projection Rules.

Rule	Type	Derivation Pattern	Weight $w(e)$
1	`app_to_app`	App/Lib `SUB` t ← `PUB` App/Lib	$\max_t w(t)$
2	`app_to_broker`	App/Lib `PUB/SUB` t ← `ROUTES` Broker	$\max_t w(t)$
3	`node_to_node`	Lifted from `app_to_app` between hosted units	Lifted $\max(w)$
4	`node_to_broker`	Lifted from `app_to_broker` for hosted units	Lifted $\max(w)$
5	`app_to_lib`	App/Lib `USES` → Library	Inherits w(app)
6	`broker_to_broker`	Colocation edge sharing physical Node	Inherits w(node)

2.3 Intrinsic QoS & Criticality Weight Propagation

SaaG assigns criticality weights $w(v) \in [0, 1]$ across all entities via an expert-elicited linear weighting of QoS contracts, normalized message size, and topology fan-out:

(1) **Intrinsic Topic Weight Formula:** For each topic $t \in V_{\text{topic}}$,

$$w(t) = \max\left(0.01,\ \beta \cdot \text{QoS\_score}(t) + (1 - \beta) \cdot \text{size\_norm}(t)\right) \tag{2}$$

where $\beta = 0.85$, $\text{size\_norm}(t) = \min\left(\frac{\log_2(1+\text{size\_kb})}{50},\ 1.0\right)$, and:

$$\text{QoS\_score}(t) = 0.30 \cdot \text{rel} + 0.40 \cdot \text{dur} + 0.30 \cdot \text{prio} \tag{3}$$

Reliability (`RELIABLE`=1.0, `BEST_EFFORT`=0.0), Durability (`PERSISTENT`=1.0, `TRANSIENT_LOCAL`=0.5, `VOLATILE`=0.0), and Transport Priority (`CRITICAL`=1.0, `HIGH`=0.66, `LOW`=0.0).

(2) **Upward Weight Propagation:** Component weights propagate hierarchically:

$$w(a) = 0.80 \cdot \max_{t \in T(a)} w(t) + 0.20 \cdot \text{mean}_{t \in T(a)} w(t) \quad (4)$$

$$w(b) = 0.70 \cdot \max_{t \in T(b)} w(t) + 0.30 \cdot \text{mean}_{t \in T(b)} w(t) \quad (5)$$

$$w(l) = \min\left(1.0,\ \text{base\_w} \cdot (1 + \gamma \cdot \log_2(1 + DG_{\text{in}}(l)))\right) \quad (6)$$

$$w(n) = \max_{v \text{ RUNS\_ON } n} w(v) \quad (7)$$

where base_w = 0.20, $\gamma = 0.15$, and $DG_{\text{in}}(l)$ is the in-degree of library l.

2.4 Operational Usages: Pipeline Gating and Platform Refactoring

The architectural digital model serves two complementary functions across the naval combat system lifecycle:

1. Pre-Deployment Verification & Pipeline Gating (Forward Mode): During concurrent CI/CD pipeline builds, SaaG instantiates an isolated candidate graph $G_{u'} = (V', E')$ by substituting candidate unit version u' into the baseline platform inventory, guaranteeing baseline immutability. The runner audits $G_{u'}$ against static middleware rules (§3) in < 1.2 s, enforcing fail-closed gating on safety-critical S1/S2 violations.

2. Architectural Improvement & Platform Refactoring (Diagnostic Mode): Beyond release gating, system architects query the global digital model (G_{baseline}) to drive systemic refactoring and technical debt reduction across fielded platforms:

- *Cycle Untangling:* Tarjan's SCC algorithm extracts cyclic subgraphs across legacy packages (e.g., WASA allocation vs threat evaluation loops), guiding architects in decoupling monolithic dependencies into clean layered abstractions.
- *Criticality Hotspot Discovery:* Combining topology fan-out with intrinsic weights ($w(v)$) pinpoints overloaded brokers and high-fanout topics requiring architectural replication or channel decomposition.
- *Dead Topic & Schema Drift Pruning:* Auditing orphaned topics ($|P(t)| = 0$ or $|S(t)| = 0$) and unreferenced libraries ($DG_{\text{in}}(l) = 0$) enables systemic cleanup of accumulated legacy technical debt.
- *"What-If" Refactoring Simulation:* Architects simulate proposed structural changes (e.g., migrating applications across hosts, altering QoS durability from `VOLATILE` to `TRANSIENT_LOCAL`) directly on the graph model, verifying system-wide safety and latency impact prior to modifying production source code.

3 Model-Driven Verification Rules & CI/CD Pipeline Gating

SaaG enforces static compliance rules across $G_{u'}$. In accordance with our modality contract:

- **Implemented in Prototype (SaaG-P):** Audited statically on descriptors.
- ○ **Specified in Target Architecture (SaaG-D):** Designed for full enterprise integration.

Table 2: Verification Rules Catalog (• Implemented; ○ Specified).

Policy / Rule	SaaG-P	SaaG-D	Target Middleware Check
Pub/Sub RxO QoS Matching	•	•	Durability & Reliability ($O \geq R$)
Middleware Pre-Conditions	•	•	`PARTITION` & `DOMAIN_ID` match
Extended Pub/Sub RxO	○	•	`DEADLINE`, `LIVELINESS`, `OWNERSHIP`
CPU Core Allocation	•	•	Core count $\leq C(v_p)$ per host
CPU Core Non-Overlap	○	•	Pairwise non-overlapping masks
Topic Continuity	•	•	Orphaned topics ($\lvert P\rvert = 0 \vee \lvert S\rvert = 0$)
Tactical Schema Match	•	•	TDL / Link 16 / STANAG match
Tactical Field AST	○	•	Bit-level payload ICD alignment
Circular Dependencies	•	•	Tarjan's SCC on dependencies
Telemetry Drift	○	•	$E_{\text{Observed}} \setminus E_{\text{Designed}}$ via logs

3.1 Severity Rubric & Alignment with Military Safety Standards (MIL-STD-882E)

To integrate cleanly with military system safety regulations (MIL-STD-882E [2]), rule violations are categorized into four severity levels (S1–S4), decoupled from message transport priority:

- **S1 (Critical Severity — Catastrophic / Safety Critical):** Flaws causing complete loss of safety-critical functions (e.g., pub/sub RxO mismatch on weapon assignment or primary radar tracking channels, CPU over-allocation on fire control servers).
- **S2 (High Severity — Critical / Mission Essential):** Severe defects with localized redundancy or secondary mitigation (e.g., orphaned tactical data link topics, circular dependencies among core WASA engagement planning units).
- **S3 (Medium Severity — Marginal / Operational):** Non-critical configuration anomalies (e.g., unassigned transport priority hints, best-effort auxiliary sensor stream topic leaks).
- **S4 (Low Severity / Informational):** Style and maintenance warnings (e.g., deprecated utility library versions, non-standard topic naming).

3.2 CI/CD Runner Integration: Exit Codes, Delta-Aware Gating, & Waiver Register

SaaG integrates into standard Jenkins and GitLab CI/CD runners via a CLI gate:

$$\text{Exit} = \begin{cases} 0, & \text{PASS: No S1/S2 violations (S3/S4 logged)} \\ 1, & \text{FAIL: S1 (Critical) or S2 (High) detected} \\ 2, & \text{ERROR: Execution / parse failure} \end{cases} \quad (8)$$

Standard CI runners treat non-zero exit codes as build failures. For safety-critical release branches (weapon control, fire authorization, track fusion), SaaG operates in a **fail-closed** configuration (Exit 2 aborts the pipeline).

3.2.1 Delta-Aware Gating & Waiver Register. In legacy naval codebases carrying pre-existing technical debt, an absolute gate blocks every build if baseline violations exist. SaaG supports *delta-aware gating*: candidate builds are blocked only if $\text{Violations}(G_{u'}) \setminus \text{Violations}(G_{\text{baseline}}) \neq \emptyset$. Known legacy violations are managed via a Chief Architect-approved *Waiver Register* with cryptographic signatures and expiration dates.

4 Multi-Scale Naval Platform Architectures & Verification Methodology

To assess the practical applicability of SaaG pre-deployment verification in an operational naval combat systems environment, we model the architectural scales and middleware configurations across five major ADVENT platform families (**ADVENT Martı**, **ADVENT Ufuk**, **ADVENT Rota**, **ADVENT Kalyon / Milgem CMS**, and **LHD CMS Flagship**).

4.1 Multi-Domain Combat System Architecture & Platform Profiles

Modern naval combat suites deploy across diverse physical and operational domains, as reflected in the public product tree of HAVELSAN ADVENT CMS:

- **Airborne Command & Control (ADVENT Martı):** Deployed on Maritime Patrol Aircraft (MPA), helicopters, and naval UAVs, handling airborne radar, sonobuoy acoustic processing, and tactical data link relays with tight embedded single-node footprints ($|V| = 1,498$, $|E| = 4,824$).
- **Coastal Surveillance & C2 Station (ADVENT Ufuk):** Land-based multi-sensor compilation center integrating coastal radar networks, AIS, ADS-B, and electro-optical surveillance across 23 distributed server nodes ($|V| = 2,174$, $|E| = 7,142$).
- **Unmanned & Autonomous Systems (ADVENT Rota):** Mission Management System for unmanned surface, underwater, and aerial platforms (USV/UUV/UAV) supporting STANAG 4586 compliance [3] and autonomous payload control ($|V| = 2,192$, $|E| = 6,980$).
- **Surface Combatant Core (Milgem CMS / ADVENT Kalyon):** Comprehensive multi-warfare suite (AAW, ASuW, ASW, EW) deployed on corvettes and frigates across 21 tactical console nodes ($|V| = 3,254$, $|E| = 11,460$).
- **Task Group Command Flagship (LHD CMS):** Fleet flagship suite (such as L400 TCG Anadolu) featuring large-scale multi-link gateway centers, joint operations planning, and 66 physical operator consoles and tactical server chassis ($|V| = 2,775$, $|E| = 9,620$).

Across all platform scales, subsystems rely on **Genieware** publish/subscribe middleware to facilitate real-time sensor-to-shooter coordination and Network Enabled Capability (NEC).

4.2 Continuous Verification & Pipeline Gating Workflow

SaaG operates directly within continuous integration runners (Jenkins, GitLab CI) upon every merge request:

1. **Descriptor Ingestion & Candidate Isolation:** The pipeline extracts candidate unit descriptors, IDL schemas, topic binding specifications, and node affinity masks, generating an isolated candidate multigraph $G_{u'}$.
2. **Failure-Dependency Projection:** Inverted failure edges ($B \xrightarrow{\text{DEPENDS\_ON}} A$) are computed along with hierarchical criticality weights.
3. **Multi-Policy Rule Evaluation:** The verification engine evaluates pub/sub QoS contracts, core pinning bounds, topic connectivity, and SCC cycles.
4. **Delta-Aware Gating:** The runner checks new violations against the baseline register and waiver list, returning Exit 0 (Pass) or Exit 1 (Fail).

4.3 Evaluation on Representative Architectural Misconfiguration Scenarios

To validate the verification engine across mission-critical middleware failure modes without relying on restricted proprietary records, we evaluate SaaG against representative classes of architectural misconfigurations:

- **Weapon Channel Durability Mismatch (Scenario A — S1):** Altering the durability of `weapon.assignment.cmd` to `VOLATILE` while the subscriber requests `TRANSIENT_LOCAL`. In pub/sub middleware (Genieware/DDS), this silently drops weapon orders. SaaG flags this in < 0.05 s.
- **OPCON CPU Core Pinning Contention (Scenario B — S1):** Overlapping CPU core affinity masks between track fusion and 3D tactical display servers on host console processors, causing latency jitter exceeding hard 25 ms deadlines.
- **Orphaned Tactical Data Link Topics (Scenario C — S2):** Refactored Link 16 J-series topics left without active subscribers ($|S(t)| = 0$), silently severing cross-platform track feeds.
- **WASA Engagement Planning Dependency Cycles (Scenario D — S2):** Transitive cycles between weapon allocation planners and threat evaluators that induce initialization deadlocks.

5 Computational Performance & Scalability of Digital Model Auditing

5.1 Multi-Scale Naval Platform Benchmarking Setup

We evaluated SaaG verification latency across 5 scaling operational profiles modeled on real-world ADVENT CMS platform configurations:

1. **ADVENT Rota (USV/UxV Platform):** 258 Apps, 1,852 Topics, 1 Node, 81 Libraries ($|V| = 2,192$, $|E| = 6,980$).

Table 3: Pre-Deployment Verification Performance Across Representative Naval Middleware Misconfiguration Scenarios.

Scenario	Architectural Fault	Sev.	Detected Rule	Time
Scen. A	Pub `VOLATILE` vs Sub `TRANSIENT_LOCAL`	S1	Pub/Sub RxO Match	0.048 s
Scen. B	CPU core overlap (cores 2–3)	S1	Core Pinning	0.052 s
Scen. C	Orphaned Link 16 forwarding topic	S2	Topic Continuity	0.038 s
Scen. D	Cyclic dependency (WASA vs threat-eval)	S2	Tarjan SCC	0.041 s
Scen. E	`PARTITION` mismatch on sensor gateway	S1	Middleware Pre-Cond.	0.039 s

(2) **ADVENT Martı (Maritime Patrol Aircraft / Helicopter):** 238 Apps, 1,219 Topics, 1 Node, 40 Libraries ($|V| = 1,498$, $|E| = 4,824$).
(3) **ADVENT Ufuk (Coastal Surveillance & C2 Station):** 184 Apps, 1,857 Topics, 23 Nodes, 110 Libraries ($|V| = 2,174$, $|E| = 7,142$).
(4) **Milgem CMS / ADVENT Kalyon (Corvette/Frigate Combatant):** 318 Apps, 2,831 Topics, 21 Nodes, 84 Libraries ($|V| = 3,254$, $|E| = 11,460$).
(5) **LHD CMS / ADVENT Kalyon (Task Group Command Flagship):** 308 Apps, 2,303 Topics, 66 Nodes, 98 Libraries ($|V| = 2,775$, $|E| = 9,620$).

Environment: Benchmarks executed on Ubuntu 22.04 LTS (AMD EPYC 7763, 32 GB RAM) using Python 3.11 with NetworkX 3.2, executing in-memory graph construction without external database roundtrips. Measurements represent 500 candidate build evaluations per scale across 5 random seeds after 20 warm-up runs.

Table 4: Gate Execution Latency Across 5 ADVENT CMS Profiles ($N = 500$).

Platform	Domain	Comp.	Graph (s)	Rules (s)	Total (s)	P95
Martı	Air C2	1,498	0.123	0.002	**0.125**	0.150
Ufuk	Coastal C2	2,174	0.406	0.003	**0.410**	0.435
Rota	USV/UxV	2,192	0.430	0.005	**0.435**	0.450
LHD CMS	Flagship	2,775	1.138	0.014	**1.152**	1.161
Milgem	Frigate	3,254	0.757	0.003	**0.760**	0.780

5.2 Scaling Dynamics & Physical Node Distribution Impact

The benchmark results in Table 4 demonstrate two central insights:

- **Dominance of Graph Construction:** Graph construction accounts for **98.4% to 99.6% of total wall-clock time** across all platforms, while pure rule auditing executes in merely **2 to 14 milliseconds** (0.002–0.014 s).
- **Impact of Physical Node Topology:** Comparing LHD CMS (2,775 components, 66 physical nodes) and Milgem CMS (3,254 components, 21 physical nodes) reveals that node count significantly impacts graph lifting. Evaluating pairwise colocation and routing dependencies across 66 OPCON consoles and server blades incurs $O(k^2)$ lifting overhead per host node, resulting in 1.152 s latency on LHD compared to 0.760 s on Milgem.

Across all operational configurations, total pipeline gating latency remains ≤ 1.15 s ($< 0.22\%$ of a typical 9-minute automated build).

6 Industrial Lessons: The Data-Acquisition Bottleneck in Architectural Digital Modeling

The primary practical insight from developing and deploying SaaG in naval defense engineering is that **the computational cost of graph analysis is negligible, but configuration data acquisition across enterprise and security silos is the true blocker.**

6.1 The Anatomy of Configuration Data Silos in Defense Projects

As summarized in Table 5, **6 of the 7 specified capabilities** were obstructed by configuration data silos:

(1) **Classified & OEM Siloing:** Application code resided in Git, while sensor/weapon Interface Control Documents (ICDs) and hardware core pinnings resided in isolated shipyard commissioning archives.
(2) **Format Fragmentation:** QoS contracts were split across XML profiles, C++ pragmas, and startup scripts.
(3) **Absence of CMDB APIs:** Enterprise asset databases were structured for manual inventory audits rather than automated CI/CD querying.

7 Related Work

Architecture Conformance & Reflexion Models: Murphy et al. [8] pioneered Software Reflexion Models to compare designs against extracted source-code call graphs. Perry and Wolf [9] and de Silva and Balasubramaniam [10] characterized architectural erosion. Terra and Valente [11] introduced dependency constraint languages. Yigit and Buzluca [17] formulated graph-based dependency analysis for critical components in pub/sub systems. SaaG builds upon these foundations and operationalizes an architectural digital model for pre-deployment CI/CD gating in naval combat management systems.

Configuration Error Detection: Xu and Zhou [12] provided a taxonomy of configuration error detection. Tang et al. [13] described holistic configuration management at Facebook. Huang et al. [14] developed ConfValley using declarative constraints. SaaG extends configuration verification to safety-critical naval combat pub/sub systems under MIL-STD-882E [2].

Policy-as-Code & Digital Twins: Policy engines (ArchUnit, OPA-Gatekeeper) enforce static rules on manifests. Kritzinger et al. [7] established the taxonomy distinguishing digital models, shadows, and twins. Tao et al. [16] reviewed industrial digital twins. SaaG adapts the digital model concept to software architecture, establishing a static, reproducible baseline for pre-deployment gating.

Table 5: Specified vs. Implemented Gap Analysis in SaaG Verification Capability for Naval Combat Systems.

Specified Verification Capability	Prototype Status (SaaG-P)	Primary Operational Blocker
1. Endpoint-Level Pub/Sub RxO Conformance	Partially Realized (Topic-level)	**Data Ingestion:** Endpoint IDL descriptors stored in vendor-siloed toolchains
2. Field-Level Payload Schema Alignment	Partially Realized (Topic name match)	**Data Ingestion:** Third-party sensor/weapon ICD parsers not integrated into build runner
3. Hardware Core Pinning Non-Overlap	Partially Realized (Host core capacity)	**Data Ingestion:** OS core-binding scripts reside outside application source repos
4. OS Memory & Kernel Parameter Audit	Partially Realized (Config requirement)	**Data Ingestion:** Target deployment host profiles not accessible via programmatic CMDB
5. Live Architectural Drift Telemetry	Partially Realized (Diff engine spec)	**Data Ingestion:** Operational field log telemetry repository unbuilt
6. Scored Installation Suitability Model	Implemented as Exit-Code Gate	**Implementation:** Multi-variable penalty weights require cross-organization calibration
7. Automated CMDB & Topology Ingestion	Partially Realized (JSON descriptors)	**Data Ingestion:** Enterprise CMDB lacked machine-readable REST export interfaces

8 Conclusion & Evolution Toward Runtime Digital Shadows

We presented **SaaG**, an architectural digital model framework for pre-deployment verification, CI/CD gating, and continuous architectural refactoring in mission-critical Naval Combat Management Systems (ADVENT CMS). Operating on an attributed multigraph ($G = (V, E, \tau_V, \tau_E, w_V, w_E)$), SaaG fulfills two complementary roles across the naval software lifecycle: (1) **Forward Gating**, automatically auditing candidate releases against publish/subscribe QoS contracts, CPU core allocations, topic continuity, and dependency cycles in < 1.2 s within CI/CD pipelines; and (2) **Diagnostic Refactoring**, providing combat system architects with global graph analytics to untangle circular package dependencies, resolve criticality bottlenecks, prune legacy topic debt, and simulate "what-if" refactorings before modifying existing operational platforms.

Benchmarks across 5 operational ADVENT profiles (from single-node embedded units like ADVENT Martı MPA at 0.125 s and ADVENT Rota USVs to the 66-node LHD Flagship at 1.152 s) demonstrate that verification requires ≤ 1.15 s ($< 0.22\%$ of an automated build), while reliably catching critical misconfigurations across representative fault scenarios. Our analysis demonstrates that graph analysis is computationally lightweight, whereas configuration data acquisition across defense engineering silos is the primary practical hurdle.

Future Work: We are developing automated ICD harvesters to bridge configuration data ingestion blockers, expanding delta-aware gating across multi-ship task group pipelines, and exploring LLM-assisted remediation proposals subject to mandatory human-in-the-loop safety reviews under naval software assurance guidelines.

References

[1] ARMERKOM / HAVELSAN. 2022. *ADVENT Combat Management System: Architectural Principles and Operational Capabilities*. Technical Whitepaper, HAVELSAN Inc.

[2] Department of Defense. 2012. *System Safety (MIL-STD-882E)*. DoD Standard Practice.

[3] NATO Standardization Office. 2017. *STANAG 4586: Standard Interfaces of UAV Control System (UCS) for NATO UAV Interoperability*, Edition 4.

[4] NATO Standardization Office. 2020. *STANAG 5516: Tactical Data Exchange - Link 16*, Edition 8.

[5] NATO Standardization Office. 2021. *STANAG 5522: Tactical Data Exchange - Link 22*, Edition 3.

[6] Object Management Group (OMG). 2015. *Data Distribution Service (DDS) Specification*, Version 1.4.

[7] W. Kritzinger et al. 2018. Digital Twin in manufacturing: A categorical literature review. *IFAC-PapersOnLine* 51, 11 (2018), 1016–1022.

[8] G. C. Murphy, D. Notkin, and K. J. Sullivan. 2001. Software reflexion models. *IEEE TSE* 27, 4 (2001), 364–380.

[9] D. E. Perry and A. L. Wolf. 1992. Foundations for the study of software architecture. *ACM SIGSOFT SEN* 17, 4 (1992), 40–52.

[10] L. de Silva and D. Balasubramaniam. 2012. Controlling software architecture erosion. *JSS* 85, 1 (2012), 132–151.

[11] R. Terra and M. T. Valente. 2014. A dependency constraint language. *SPE* 44, 9 (2014), 1073–1094.

[12] T. Xu and Y. Zhou. 2015. Systems approaches to tackling configuration errors. *ACM CSUR* 47, 4 (2015), 70.

[13] C. Tang et al. 2015. Holistic configuration management at Facebook. In *ACM SOSP '15*. 328–343.

[14] P. Huang et al. 2015. ConfValley: Validating system configurations. In *ACM EuroSys '15*. Article 4.

[15] J. Humble and D. Farley. 2010. *Continuous Delivery*. Addison-Wesley.

[16] F. Tao et al. 2019. Digital twin in industry: State-of-the-art. *IEEE TII* 15, 4 (2019), 2405–2415.

[17] I. O. Yigit and F. Buzluca. 2024. A Graph-Based Dependency Analysis Method for Identifying Critical Components in Distributed Publish-Subscribe Systems. *IEEE Access* (2024). https://doi.org/10.1109/ACCESS.2024